Введение в планировщик

Суть проблемы: Как разработать политики планировщика
Как должны разрабатываться базовые фреймворки политик планировщика? Какие должны быть ключевые предположения? Какие метрики важны? Какие базовые техники использовались в ранних вычислительных системах?

Предположения рабочей нагрузки

Перед тем как обсудить возможные политики, для начала сделаем несколько упрощающих отступлений о процессах, запущенных в системе, которые все вместе называются рабочей нагрузкой. Определяя рабочую нагрузку как критическую часть построения политик и чем больше вы знаете о нагрузке, тем более качественную политику вы сможете написать.

Сделаем следующие предположения о процессах запущенных в системе, иногда еще называемых jobs (задачами). Практически все эти предположения нереалистичны, но нужны для развития мысли.

1) Каждая задача запущена одинаковое количество времени
2) Все задачи ставятся одновременно
3) Поставленная задача работает до своего завершения
4) Все задачи используют только CPU
5) Время работы каждой задачи известно.

Метрики Планировщика

Кроме некоторых предположений о нагрузке необходимо еще некоторый инструмент сравнения различных политик планирования: метрики планировщика. Метрика это всего лишь некоторая мера чего-либо. Существует некоторое количество метрик которые можно использовать для сравнения планировщиков.

Для примера будем использовать метрику, которая называется время оборота (turnaround time). Время оборота задачи определяется как разница между временем завершения задачи и время поступления задачи в систему.

$$T_{turnaround} = T_{completion} - T_{arrival}$$

Поскольку мы предположили что все задачи поступили в одно время, то Ta=0 и таким образом Tt=Tc. Это значение естественно поменяется когда мы изменим вышеперечисленные предположения.

Другая метрика - fairness (справедливость, честность). Производительность и честность часто являются противодействующими характеристиками в планировании.

Например планировщик может оптимизировать производительность, но ценой ожидания запуска другими задачами, таким образом снижается честность.

FIRST IN FIRST OUT (FIFO)

Самый базовый алгоритм, который мы можем воплотить называется FIFO или first come, first served. Этот алгоритм имеет несколько преимуществ: он очень прост в реализации и он подходит под все наши предположения, выполняя работу довольно хорошо.

Рассмотрим простой пример. Допустим 3 задачи были поставлены одновременно. Но предположим что задача А пришла на волосок раньше всех остальных, поэтому в списке выполнения будет стоять раньше остальных, точно также как Б относительно В. Предположим что каждая из них будет выполняться 10 секунд. Какой в этом случае будет среднее время выполнения этих задач? (рис)

Подсчитав значения - 10+20+30 и разделив на 3 получим среднее время выполнения программы 20 секунд.

Теперь попробуем изменить наши предположения. В частности предположение 1 и таким образом не будем более предполагать что каждая задача исполняется одинаковое количество времени. Как покажет себя FIFO в этот раз?

Как оказывается разные времена исполнения задач крайне негативно сказываются на продуктивности алгоритма FIFO. Предположим что задача А будет исполняться 100 секунд, тогда как Б и В по-прежнему будут по 10 каждая. (рис)

Как видно из рисунка, среднее время для системы получится (100+110+120)/3=110. Такой эффект называется эффектом конвоя, когда некоторые кратковременные потребители какого-либо ресурса будут стоять в очереди вслед за тяжелым потребителем. Это похоже на очередь в продуктовом магазине, когда перед вами покупатель с полной тележкой. Лучшее решение проблемы - попытаться сменить кассу или же расслабиться и дышать глубоко.

Shortest Job First

Можно ли как-то решить подобную ситуацию с тяжеловесными процессами? Конечно. Другой тип планирования называется SJF. Алгоритм его также достаточно примитивен - как понятно из названия, первыми будут запущены самые короткие задачи одна за другой. (рис)

В данном примере результатом запуска тех же самых процессов будет улучшение среднего времени оборота программ и оно будет равно 50 вместо 110, что практически в 2 раза лучше.

Таким образом для заданного предположения что все задачи прибывают в одно и тоже время, алгоритм SJF кажется наиболее оптимальным алгоритмом. Однако наши предположения все еще не кажутся реалистичными. В этот раз изменим предположение 2 и в этот раз представим что задачи могут пребывать в любое время, а не все одновременно. К каким проблемам это может привести?

Представим что задача А (100с) прибывает самой первой и начинает исполняться. В момент t=10 прибывают задачи Б,В, каждая из которых будет занимать 10 секунд. Таким образом среднее время выполнения это (100+(110-10)+(120-10))\3 = 103. Что бы мог планировщик сделать чтобы улучшить положение?

Shortest Time-to-Completion First (STCF)

Для того чтобы улучшить положение опустим предположение 3 о том что программа запущена и работает до завершения. Кроме того нам понадобится поддержка оборудования и как вы могли догадаться мы будем использовать таймер для прерывания работающей задачи и переключения контекстов. Таким образом планировщик может что-то предпринять в момент поступления задач Б,В - прекратить выполнение задачи А и поставить в обработку задачи Б и В и после их окончания продолжить выполнение процесса А. Такой планировщик называется STCF или Preemptive Job First. (рис)

Результатом работы этого планировщика будет такой результат: ((120-0)+(20-10)+(30-10))/3=50. Таким образом такой планировщик становится еще более оптимальным для наших задач.

Метрика Время отклика (Response Time)

Таким образом, если мы знаем время работы задач и то, что эти задачи используют только CPU, STCF будет наилучшим решением. И когда-то в ранние времена эти алгоритмы работали и довольно-таки неплохо. Однако теперь пользователь проводит основное время за терминалом и ожидает от нее производительного интерактивного взаимодействия. Так родилась новая метрика - время ответа (отклика).

Время отклика считается следующим образом:

$$T_{response} = T_{firstrun} - T_{arrival}$$

Таким образом для предыдущего примера время отклика будет таким: А=0, Б=0, В=10 (abg=3,33).

И оказывается алгоритм STCF алгоритм не так уж и хорош в ситуации, когда 3 задачи прибудут одновременно - ему придется дождаться пока маленькие задачи полностью не завершатся. Таким образом алгоритм хорош для метрики времени оборота, но плох для метрики интерактивности. Представьте что сидя за терминалом в попытке напечатать символы в редакторе вам пришлось бы ждать более 10 секунд, потому что какая-то другая задача занимает процессор. Это не очень-то и приятно.

Таким образом мы сталкиваемся с другой проблемой - как мы можем построить планировщик, который бы был чувствителен к времени отклика?

Round Robin

Для решения этой проблемы был разработан алгоритм RR. Основная идея довольно проста: вместо запуска задач до полного завершения, будем запускать задачу на некоторый промежуток времени (называемый квантом) и затем переключаться на другую задачу из очереди. Алгоритм повторяет свою работу до тех пор пока все задачи не завершатся. При этом время работы программы должно быть кратен времени через которое таймер прервет процесс. Например, если таймер прерывает процесс каждые x=10мс, то размер окна выполнения процесса должен быть кратен 10 и быть 10,20 или x*10.

Рассмотрим пример: Задачи АБВ прибывают одновременно в систему и каждая из них хочет работать 5 секунд. Алгоритм SJF будет выполнять каждую задачу до конца перед тем как запустить другую. В противопоставление алгоритм RR с окном запуска=1с пройдется по задачам следующим образом (рис):

Среднее время отклика для алгоритма RR (0+1+2)/3=1, тогда как для SJF (0+5+10)/3=5.

Логично предположить, что окно времени очень важный параметр для RR, чем меньше оно, тем выше время ответа. Однако, нельзя делать его и чересчур маленьким, поскольку время на переключение контекста будет также играть свою роль в общей производительности. Таким образом выбор времени окна исполнения выставляется архитектором ОС и зависит от задач, которые в ней планируются исполняться. Переключение контекста не единственная служебная операция которая тратит время - запущенная программа много чем еще оперирует, например различными кешами и при каждом переключении необходимо сохранять и восстанавливать эту среду, что также может требовать много времени.

RR - отличный планировщик, если бы речь шла только о метрике времени отклика. Но как поведет себя метрика времени оборота задачи при этом алгоритме? Рассмотрим пример выше, когда время работы А,Б,В=5с и поступают в одно и то же время. Задача А завершится в 13, Б в 14, В в 15с и среднее время оборота получится 14с. Таким образом RR является самым плохим алгоритмом для метрики оборота.

Более общими словами, любой алгоритм типа RR - честный, он делит время работы на CPU поровну между всеми процессами. И таким образом эти метрики постоянно конфликтуют между собой.

Таким образом у нас есть несколько противопоставленных алгоритмов и при этом ее остается несколько предположений - о том что время задачи известно и что задача только использует CPU.

Смешение с I/O

В первую очередь уберем предположение 4 что процесс только использует CPU, естественно это не так и процессы могут обращаться к другому оборудованию.

В момент, когда какой-либо процесс запрашивает операцию ввода-вывода, процесс переходит в состояние blocked, ожидая завершения I/O. Если I/O посылается к жесткому диску, то такая операция может занимать вплоть до нескольких мс или дольше, и процессор в этот момент будет простаивать. В это время планировщик может занять процессор любым другим процессом. Следующее решение которое придется принимать планировщику - когда процесс завершит свой I/O. Когда это случится произойдет прерывание и ОС переведет вызвавший I/O процесс в состояние ready.

Рассмотрим пример из нескольких задач. Каждая из них нуждается в 50мс процессорного времени. Однако первая будет каждые 10мс обращаться к I/O (который тоже будет исполняться по 10мс). А процесс Б просто использует 50мс процессор без I/O. (рис)

В данном примере будем использовать STCF планировщик. Как поведет себя планировщик, если запустить на нем такой процесс как А? Он поступит следующим образом - сначала полностью отработает процесс А, а потом процесс Б.

Традиционный подход для решения этой проблемы - интерпретировать каждую 10-мс подзадачу процесса А как отдельную задачу. Таким образом при старте с алгоритмом STJF, выбор между 50-мс задачей и 10-мс задачей очевиден. Затем когда подзадача А завершится будет запущен процесс Б и I/O. После завершения I/O будет принято снова запустить 10-мс процесс А вместо процесса Б. Таким образом возможно реализовать перекрытие, когда CPU используется другим процессом, пока первый ожидает I/O. И как итог система лучше утилизируется - в момент когда интерактивные процессы ожидают I/O на процессоре могут выполняться и другие процессы.

Оракула больше нет

Теперь попробуем избавиться о предположении, что время работы задачи известно. Это в целом самое плохое и нереальное предположение из всего списка. Фактически в среднестатистических обычных ОС, сама ОС обычно очень мало знает о времени исполнения задач, как же тогда строить планировщик без знания того сколько времени задача будет исполняться? Быть может мы могли бы использовать некоторые принципы RR для решения этой проблемы?

Итог

Мы рассмотрели базовые идеи планирования задач и рассмотрели 2 семейства планировщиков. Первый запускает кратчайшую задачу вначале и таким образом повышает время оборота, второй же разрывается между всеми задачами одинаково, повышая время отклика. Оба алгоритма плохи там, где хороши алгоритмы другого семейства. Также мы рассмотрели как параллельное использование CPU и I/O могут улучшить производительность, но так и не решили проблему с ясновидением ОС. И на следующем занятии мы рассмотрим планировщик, который смотрит в ближайшее прошлое и пытается предугадать будущее. И называется он multi-level feedback queue.